Unité 5 — Analyse de binaires & sanitizers

Livrable B — Support de cours · UE « Software Security »

Code UE 7532 14 U32 D1 · 11 octobre 2026

> **Place dans le programme.** Répond au point « identifier et expliquer les problèmes de sécurité dans le code source et les binaires » et « évaluer la sécurité d'un code ou d'une application ». On outille l'étudiant pour **analyser** un binaire sans source (statique et dynamique) et pour **instrumenter** du code avec les sanitizers. *Outils statiques/dynamiques issus de meltypot (strings, objdump, gdb, radare2) ; détail ASan/UBSan : contenu auteur.*

1. Analyse statique vs dynamique

Approche	Principe	Outils
Statique	Examiner le binaire/code *sans l'exécuter*.	`strings`, `objdump`, `radare2`, SAST (U9).
Dynamique	Observer le programme *pendant* l'exécution.	`gdb`, `strace`/`ltrace`, sanitizers, fuzzing (U11).

Les deux sont complémentaires : le statique donne la structure et les chemins possibles ; le dynamique confirme ce qui se passe réellement sur une entrée donnée.

2. Analyse statique d'un binaire

`strings` révèle les chaînes (messages, formats, chemins) — premières pistes.

STRINGS — EXTRAIT RÉEL D'UN BINAIRE DE LABO

```
Bienvenue !
usage: %s <nom>
Bonjour, %s
```

`objdump -d` désassemble et liste les fonctions ; **`radare2`** va plus loin : analyse automatique (`aa`), liste des fonctions (`afl`), références croisées (`axt`), recherche de gadgets (`/R`, `/x`).

OBJDUMP — FONCTIONS (RÉEL)

```
08049186 <func>:
08049199 <main>:
```

RADARE2 — FONCTIONS ET XREF VERS STRCPY (RÉEL)

```
[Fonctions]      afl
0x08049186    sym.func
0x08049199    main
[Xrefs vers strcpy]   axt sym.imp.strcpy
main 0x80491cc [CALL] call sym.imp.strcpy
```

Repérer un appel à une fonction dangereuse (`strcpy`, `gets`…) via les **références croisées** oriente directement vers une vulnérabilité potentielle.

3. Analyse dynamique

`gdb` — points d'arrêt, inspection de registres et de pile (cf. Unité 2). **`strace`** trace les appels système, **`ltrace`** les appels de bibliothèque : utiles pour voir ce qu'un binaire *fait* (fichiers ouverts, commandes lancées) sans son code.

4. Les sanitizers

Instrumentation à la compilation qui transforme des comportements indéfinis en abandons immédiats et localisés.

Sanitizer	Détecte	Drapeau
AddressSanitizer (ASan)	débordements pile/tas, use-after-free, double-free	`-fsanitize=address`
UndefinedBehaviorSanitizer (UBSan)	dépassement d'entier signé, décalages invalides, déréf. nul…	`-fsanitize=undefined`
MemorySanitizer / Valgrind	lectures non initialisées	`-fsanitize=memory` / `valgrind`

Exemple UBSan réel (dépassement d'entier signé) :

UBSAN — RÉEL

```
intov.c:5:24: runtime error: signed integer overflow:
    2000000000 + 1000000000 cannot be represented in type 'int'
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior intov.c:5:24
```

5. Workflow d'évaluation

1. **Reconnaissance statique** : `strings`, `iI`/`checksec` (protections ?), `afl`.
2. **Repérer les points chauds** : xrefs vers fonctions dangereuses, entrées utilisateur.
3. **Confirmer dynamiquement** : gdb/strace sur une entrée ciblée.
4. **Instrumenter** : recompiler (si source) avec ASan/UBSan, ou fuzzer (U11).
5. **Qualifier** : type (CWE), gravité, exploitabilité (U10).

6. Limites

Les sanitizers ne détectent que ce qui est **effectivement exécuté** : couverture dépendante des entrées (d'où le fuzzing). L'analyse statique de binaire est limitée par l'optimisation, le strip des symboles et l'obfuscation.

7. Points clés

- Statique (structure) et dynamique (comportement réel) se complètent.
- Les xrefs vers `strcpy`/`gets` pointent les vulnérabilités probables.
- ASan/UBSan/Valgrind donnent des diagnostics précis mais seulement sur le code exécuté.

Outils issus de meltypot ; détails ASan/UBSan auteur. Sorties strings/objdump/radare2/UBSan produites et vérifiées.